# 作业 2 答案

P4

(a) 浏览器请求的文档 URL 是什么？

第一行包含请求路径：

/cs453/index.html

Host 字段提供服务器域名：

gaia.cs.umass.edu

HTTP URL 由协议、主机名和文件路径组成：

URL=协议+主机名+路径

答案：

http://gaia.cs.umass.edu/cs453/index.html

(b) 浏览器运行的是 HTTP 的哪个版本？

第一行是：

GET /cs453/index.html HTTP/1.1

最后的 HTTP/1.1 表示使用的 HTTP 版本。

答案： HTTP/1.1。

(c) 使用非持续连接还是持续连接？

报文中包含：

Connection: keep-alive

keep-alive 表示希望保持 TCP 连接，以便后续 HTTP 请求复用。

答案： 持续连接。

(d) 浏览器所在主机的 IP 地址是什么？

HTTP 请求报文中的 Host 是目标服务器的主机名，不是浏览器所在主机的 IP 地址。源 IP 地址通常记录在 IP 数据包头部，而不是 HTTP 请求文本中。

答案： 无法仅从该 HTTP 报文确定浏览器所在主机的 IP 地址。

(e) 浏览器类型是什么？为什么需要浏览器类型？

查看 User-Agent 字段：

Mozilla/5.0 (Windows; U; Windows NT 5.1; ...) Gecko/... Netscape/7.2

可以看出它标识的是 Netscape 7.2 浏览器。HTTP 服务器可能根据不同浏览器的兼容性和功能，返回不同版本的网页内容。

答案： Netscape 7.2。User-Agent 用来告诉服务器客户端浏览器的信息，帮助服务器返回合适的内容。

P7

整个过程分为三部分。

第一步：DNS 查询

需要依次查询 n 个 DNS 服务器，每次查询耗时为对应的 RTT。

因此 DNS 查询总时间：

$$T_{DNS} = \sum_{i=1}^{n} RTT_i$$

第二步：建立 TCP 连接

HTTP 使用 TCP，需要通过三次握手建立连接。

耗时：

$$T_{TCP} = RTT_0$$

第三步：发送 HTTP 请求并接收响应

建立连接后，客户端发送 HTTP GET 请求，服务器返回对象。

由于对象非常小，可以忽略传输时间。

耗时：

$$T_{HTTP} = RTT_0$$

将三个部分相加：

$$T = T_{DNS} + T_{TCP} + T_{HTTP}$$

最终答案

$$T = \sum_{i=1}^{n} RTT_i + 2RTT_0$$

P8

(a) 非持续 HTTP，无并行连接

非持续 HTTP 每传输一个对象都需要建立新的 TCP 连接。对于单个对象，建立 TCP 连接需要：

$$1RTT_0$$

发送 HTTP 请求并接收响应需要：

$$1RTT_0$$

所以单个对象的传输时间为：

$$T_{object} = 2RTT_0$$

首先下载基础 HTML 文件：

$$T_{HTML} = 2RTT_0$$

然后顺序下载 8 个嵌入对象：

$$T_{objects} = 8 \times 2RTT_0 = 16RTT_0$$

再加上 DNS 查询时间：

$$T = \sum_{i=1}^{n} RTT_i + 2RTT_0 + 16RTT_0$$

最终答案：

$$T = \sum_{i=1}^{n} RTT_i + 18RTT_0$$

(b) 非持续 HTTP，5 条并行连接

首先下载基础 HTML 文件：

$$T_{HTML} = 2RTT_0$$

网页中包含 8 个嵌入对象。最多允许 5 条并行 TCP 连接，因此可以分成两批下载。

第一批：同时下载 5 个对象。

$$T_1 = 2RTT_0$$

第二批：同时下载剩余 3 个对象。

$$T_2 = 2RTT_0$$

因此嵌入对象的总下载时间：

$$T_{objects} = T_1 + T_2 = 4RTT_0$$

总时间：

$$T = \sum_{i=1}^{n} RTT_i + 2RTT_0 + 4RTT_0$$

最终答案：

$$T = \sum_{i=1}^{n} RTT_i + 6RTT_0$$

(c) 持续 HTTP

持续 HTTP 可以复用同一条 TCP 连接，不需要为每个对象重新建立连接。

需要分别讨论是否使用流水线。

情况一：不使用流水线（Non-pipelining）

建立 TCP 连接：

$$T_{TCP} = RTT_0$$

获取基础 HTML：

$$T_{HTML} = RTT_0$$

依次请求并获取 8 个对象：

$$T_{objects} = 8RTT_0$$

总时间：

$$T = \sum_{i=1}^{n} RTT_i + RTT_0 + RTT_0 + 8RTT_0$$

最终答案：

$$T = \sum_{i=1}^{n} RTT_i + 10RTT_0$$

情况二：使用流水线（Pipelining）

建立 TCP 连接需要：

$$RTT_0$$

获取基础 HTML 需要：

$$RTT_0$$

随后浏览器可以连续发送 8 个嵌入对象的请求，而不必等待前一个响应。

在忽略对象传输时间的情况下，所有嵌入对象的请求与响应约需要：

$$RTT_0$$

总时间：

$$T = \sum_{i=1}^{n} RTT_i + RTT_0 + RTT_0 + RTT_0$$

最终答案：

$$T = \sum_{i=1}^{n} RTT_i + 3RTT_0$$

P13

SMTP 邮件发送过程包括：

1.SMTP 协议命令。

2.邮件报文头部和正文。

1. MAIL FROM

MAIL FROM: 是 SMTP 协议中的命令。它指定邮件信封中的发件人地址，主要用于邮件传输和退信处理。

例如：

MAIL FROM:<return@example.com>

2. From

From: 是邮件报文头部中的字段。

它表示邮件的作者或收件人在邮件客户端中看到的发件人。

例如：

From: Alice <alice@example.com>

3. 两者的区别

MAIL FROM: 与 From: 分别属于 SMTP 信封和邮件内容。因此两者使用的地址不一定相同。

最终答案

MAIL FROM: 指定 SMTP 信封发件人地址，主要用于邮件传输和错误退信。

From: 指定邮件报文头部中的发件人，主要用于标识邮件作者。

两者功能不同，地址也可以不同。

P18

(a) 什么是 WHOIS 数据库？

WHOIS 是一种用于查询互联网资源注册和管理信息的服务.

WHOIS 数据库用于存储和查询域名、IP 地址以及其他互联网资源的注册和管理信息。

(b) 使用 WHOIS 数据库查询两台 DNS 服务器

可以访问 ICANN Lookup：

https://lookup.icann.org/

例如，分别查询以下域名：

google.com

cloudflare.com

在返回的注册信息中查找 Name Server 字段，记录得到的 DNS 服务器名称。

具体服务器名称需要以实际查询结果为准。

(c) 使用 nslookup 查询 DNS 服务器

首先使用默认 DNS 服务器查询：

nslookup google.com

使用指定 DNS 服务器查询：

nslookup -type=A google.com <DNS 服务器 IP>

查询 NS 记录：

nslookup -type=NS google.com

查询 MX 记录：

nslookup -type=MX google.com

查询时应分别记录三台 DNS 服务器的响应。

不同 DNS 服务器的结果可能因为缓存、配置或地理位置等因素而不同。

具体查询结果需要实际执行命令后记录。

(d) 查询具有多个 IP 地址的 Web 服务器

执行以下命令：

nslookup www.google.com

查看返回的 Address 字段。如果出现多个不同的 IP 地址，说明该域名当前解析到多个 IP 地址。

然后查询学校或公司的 Web 服务器，例如：

nslookup www.example.edu

需要将 www.example.edu 替换为真实域名。

(e) 使用 ARIN WHOIS 查询大学 IP 地址范围

访问 ARIN 查询网站：

https://search.arin.net/rdap/

输入大学名称或相关服务器 IP 地址。观察查询结果中的：

NetRange

CIDR

Organization

其中：

•NetRange：IP 地址范围。

•CIDR：CIDR 网络地址表示法。

•Organization：相关组织名称。

(f) 攻击者如何使用 WHOIS 和 nslookup 进行网络侦察？

攻击者可以通过 WHOIS 和 nslookup 收集公开的域名、DNS 服务器、邮件服务器和 IP 地址段等信息，从而初步了解目标机构的网络结构。

(g) 为什么 WHOIS 数据库应该公开？

WHOIS 数据库应提供适当的公开访问，以支持网络管理、故障排查、安全事件处理和互联网资源管理。

同时，应对敏感个人信息进行必要的隐私保护。

P23

定义：

F：文件大小。

N：用户数量。

$u_s$：服务器上传速率。

$u_i$：第 i 个用户的上传速率。

$d_{min}$：所有用户中的最小下载速率。

$D_{P2P}$：P2P 架构的最小文件分发时间。

(a) 当 $u_s/N \le d_{min}$ 时，证明存在分发时间为 $NF/u_s$ 的方案。

服务器的总上传速率为：$u_s$

将上传带宽平均分配给 N 个客户端，每个客户端可获得：

$$r = \frac{u_s}{N}$$

根据题目条件：

$$\frac{u_s}{N} \le d_{min}$$

说明所有客户端都有足够的下载带宽接收数据。

单个客户端下载大小为 F 的文件需要：

$$D = \frac{F}{r}$$

代入：

$$D = \frac{F}{u_s/N}$$

整理得到：

$$D = \frac{NF}{u_s}$$

最终答案：

$$\boxed{D_{cs} = \frac{NF}{u_s}}$$

(b) 当 $u_s/N \geq d_{min}$ 时，证明存在分发时间为 $F/d_{min}$ 的方案。

由条件：

$$\frac{u_s}{N} \geq d_{min}$$

可得：

$$u_s \geq Nd_{min}$$

因此服务器具有足够的上传带宽，可以同时向所有客户端以 $d_{min}$ 的速率发送文件。

每个客户端下载完整文件所需时间不超过：

$$D = \frac{F}{d_{min}}$$

其中最慢客户端的下载能力是限制因素。

最终答案：

$$\boxed{D_{cs} = \frac{F}{d_{min}}}$$

(c) 推导最小文件分发时间

客户端—服务器架构存在两个限制。

限制一：服务器上传能力

服务器需要向 N 个客户端分别上传完整文件。

服务器总共需要上传：

$$NF$$

因此：

$$D_{cs} \geq \frac{NF}{u_s}$$

限制二：最慢客户端下载能力

每个客户端都必须下载完整文件。

最慢客户端的下载时间至少为：

$$D_{cs} \geq \frac{F}{d_{min}}$$

因此最小分发时间必须同时满足这两个约束：

$$D_{cs} \geq \max\left\{\frac{NF}{u_s}, \frac{F}{d_{min}}\right\}$$

根据 (a) 和 (b) 所构造的方案，可以达到该下界

最终答案

$$\boxed{D_{cs} = \max\left\{\frac{NF}{u_s}, \frac{F}{d_{min}}\right\}}$$

P24

(a) 当 $u_s \leq \frac{u_s+u_1+\cdots+u_N}{N}$，证明存在分发时间为：$\frac{F}{u_s}$ 的方案。

第一步：确定服务器上传限制

服务器是文件的初始拥有者，为了让文件能够被其他用户获得，服务器至少需要上传一份完整文件的数据。

因此：

$$D_{P2P} \geq \frac{F}{u_s}$$

第二步：计算系统总上传速率

服务器和所有用户均可以参与上传。

定义总上传速率：

$$U = u_s + \sum_{i=1}^{N} u_i$$

由题目条件：

$$u_s \le \frac{U}{N}$$

可得：

$$Nu_s \le U$$

第三步：构造分发方案

服务器以速率 $u_s$ 上传文件。用户在获得数据后，可以将已获得的文件块继续分发给其他用户。

由于系统总上传能力满足：

$$U \ge Nu_s$$

因此在理想流体分发模型下，通过用户之间的协同上传，可以在服务器上传完一份文件的时间内，将完整文件分发给所有用户。

所需时间：

$$D_{P2P} = \frac{F}{u_s}$$

最终答案：

$$\boxed{D_{P2P} = \frac{F}{u_s}}$$

(b) 当 $u_s \ge \frac{u_s+u_1+\cdots+u_N}{N}$ ，证明存在分发时间为：$\frac{NF}{u_s+u_1+\cdots+u_N}$ 的方案。

第一步：计算总数据量

每个用户都需要获得完整文件，因此 N 个用户总共需要接收：

$$NF$$

比特的数据。

第二步：计算总上传能力

服务器和用户的总上传速率：

$$U = u_s + \sum_{i=1}^{N} u_i$$

因此根据上传带宽限制：

$$D_{P2P} \ge \frac{NF}{U}$$

第三步：检查服务器上传限制

根据题目条件：

$$u_s \geq \frac{U}{N}$$

所以：

$$Nu_s \geq U$$

等价于：

$$\frac{F}{u_s} \leq \frac{NF}{U}$$

说明服务器上传一份完整文件所需时间，不超过总上传能力所决定的分发时间。

第四步：构造协同分发方案

在理想流体模型下，服务器与所有用户可以协同上传文件数据，通过合理分配各节点的上传任务，使总上传能力得到充分利用。

因此可以达到：

$$D_{P2P} = \frac{NF}{U}$$

代入 U：

$$D_{P2P} = \frac{NF}{u_s + \sum_{i=1}^{N} u_i}$$

最终答案：

$$\boxed{D_{P2P} = \frac{NF}{u_s + \sum_{i=1}^{N} u_i}}$$

(c) 推导一般情况下的最小分发时间

P2P 文件分发存在两个主要限制。

限制一：服务器上传能力

服务器至少需要上传一份完整文件：

$$D_{P2P} \geq \frac{F}{u_s}$$

限制二：整个 P2P 系统的总上传能力

所有用户总共需要接收：NF 比特的数据。

系统总上传速率为：

$$u_s + \sum_{i=1}^{N} u_i$$

因此：

$$D_{P2P} \geq \frac{NF}{u_s + \sum_{i=1}^{N} u_i}$$

由于题目假设客户端的下载带宽足够大，暂时不需要考虑最慢客户端的下载约束。

取两个下界中的较大值：

最终答案

$$\boxed{D_{P2P} = \max\left\{\frac{F}{u_s}, \frac{NF}{u_s + \sum_{i=1}^{N} u_i}\right\}}$$

P26

(a) Bob 能否在不上传的情况下获得完整文件？

BitTorrent 使用多种机制控制节点之间的文件块交换。

机制一：Tit-for-Tat（以牙还牙）

BitTorrent 通常优先向能够提供上传数据的对等方发送数据，如果 Bob 完全不上传数据，他通常无法通过正常的互惠机制持续获得优先上传服务。

机制二：Optimistic Unchoking（乐观疏通）

BitTorrent 会周期性地选择一个对等方进行上传，即使该对等方此前没有向自己提供数据。

因此，Bob 即使没有上传数据，也可能获得其他节点提供的文件块。

经过较长时间后，Bob 可能逐渐收集到完整文件。

最终答案：

Bob 有可能在不上传数据的情况下获得完整文件，但下载速度通常较慢，也不能保证一定成功。原因是 BitTorrent 的 Optimistic Unchoking 机制允许节点偶尔向未提供上传数据的其他节点发送文件块。

(b) Bob 如何利用多台具有不同 IP 地址的计算机提高搭便车效率？

假设 Bob 可以使用多台具有不同 IP 地址的计算机。

他可以让这些计算机分别作为独立的 BitTorrent 对等方加入同一个 swarm，每台计算机都有机会被其他节点通过 Optimistic Unchoking 机制选中。

因此，与只使用一台计算机相比，多个对等方可能带来更多获取数据的机会。

例如：

| 计算机 | 下载的文件块 |
| --- | --- |
| Computer A | Blocks 1, 2, 3 |
| Computer B | Blocks 4, 5, 6 |
| Computer C | Blocks 7, 8, 9 |

Bob 可以收集这些计算机获得的文件块，然后将其组合为完整文件。实际效果受到网络状况、节点选择机制和文件块重复下载等因素影响。

最终答案：

Bob 可以使用多台具有不同 IP 地址的计算机，分别作为独立节点加入同一个 BitTorrent swarm。

这样可以增加获得 Optimistic Unchoking 上传机会的可能性。

最后将不同计算机下载的文件块合并，从而提高获取完整文件的效率。

已批改作业：

https://web.ugreen.cloud/web/#/share/99c08f1f2388485d9d6e60132b7c4f78

提取码：F5L5